```
      "type":"patient",
      "id":1
    }
  }
}
```

To fetch the patients belonging to a doctor, we use a `search` query called `parent_id` that expects the child type (`patient`) and the parent's ID (Dr. Montgomery's document ID is 1). This query returns Dr. Montgomery's patients: Mr. and Mrs. Doe.

> **NOTE** Implementing parent-child relationships in Elasticsearch has performance implications. As we discussed in chapter 1, Elasticsearch may not be the right tool if you are considering document relationships, so use this feature judiciously.

### 4.6.6 The search_as_you_type data type

Most search engines suggest words and phrases as we type in a search bar. This feature has various names: search as you type, typeahead, autocomplete, or suggestions. Elasticsearch provides a convenient data type called `search_as_you_type` to support this feature. Behind the scenes, Elasticsearch works very hard to ensure that fields tagged as `search_as_you_type` are indexed to produce n-grams, which we see in action in this section.

Suppose we are asked to support typeahead queries on a `books` index: when the user starts typing a book's title letter by letter in a search bar, we should be able to suggest books based on the letters they type. First we need to create a schema with the field of type `search_as_you_type`.

**Listing 4.40 Mapping schema with the `search_as_you_type` type**

```
PUT tech_books4
{
  "mappings": {
    "properties": {
      "title": {
        "type": "search_as_you_type"
        }
     }
  }
}
```

The title supports the typeahead feature.

The notable point is that the title in the schema definition is declared as the `search_as_you_type` data type. Let's index a few documents with various titles (this book plus my—hopefully—future titles).

**Listing 4.41 Indexing a few books**

```
PUT tech_books4/_doc/1
{
  "title":"Elasticsearch in Action"
}
```